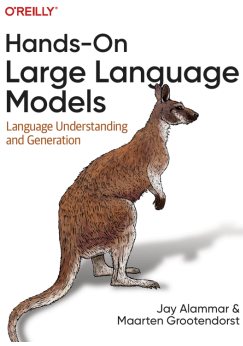
杰伊·阿拉马尔 (/)
一次一个概念地可视化机器学习。
阅读我们的书《动手大型语言模型 (https://www.LLM-book.com)》，并在LinkedIn (https://www.linkedin.com/in/jalammar/)、Bluesky (https://bsky.app/profile/jayalammar.bsky.social)、Substack (https://newsletter.languagemodels.co/)、X (https://x.com/JayAlammar)、YouTube (https://www.youtube.com/channel/UCmOwsoHty5PrmE-3QhUBfPQ)上关注我

图解变形金刚

讨论：黑客新闻（65分，4条评论），(https://news.ycombinator.com/item?id=18351674)Reddit r/MachineLearning（29分，3条评论）(https://www.reddit.com/r/MachineLearning/comments/8uh2yz/p_the_illustrated_transformer_a_visual_look_at/)
翻译：阿拉伯语 (https://www.mundhor.site/post/post14)，中文（简体）1 (https://blog.csdn.net/yujianmin1990/article/details/85221271)，中文（简体）2 (https://blog.csdn.net/qq_36667170/article/details/124359818)，法语1 (https://a-coles.github.io/2020/11/15/transformer-illustre.html)，法语2 (https://lbourdois.github.io/blog/nlp/Transformer/)，意大利语 (https://medium.com/@val.mannucci/il-transformer-illustrato-it-37a78e3e2348)，日语 (https://tips-memo.com/translation-jayalmmar-transformer)，韩语，波 (https://nlpinkorean.github.io/illustrated-transformer/)斯语 (http://dml.qom.ac.ir/2022/05/17/illustrated-transformer/)，俄语 (https://habr.com/ru/post/486358/)，西班牙语1 (https://www.ibidemgroup.com/edu/transformer-ilustrado-jay-alammar/)，西班牙语2 (https://hackernoon.com/el-transformador-ilustrado-una-traduccion-al-espanol-0y73wwp)，越南语 (https://trituenhantao.io/tin-tuc/minh-hoa-transformer/)
观看：麻省理工学院的深度学习“最先进 (https://youtu.be/53YvP6gdD7U?t=432)”讲座，引用本文
曾在斯坦福 (https://web.stanford.edu/class/cs224n/)、哈佛 (https://scholar.harvard.edu/binxuw/classes/machine-learning-scratch/materials/transformers)、麻省理工 (https://ocw.mit.edu/courses/6-s897-machine-learning-for-healthcare-spring-2019/d39a6eed387ee90b1f72a01949c35c7b_MIT6_S897S19_lec8.pdf)、普林斯顿 (https://www.cs.princeton.edu/courses/archive/fall22/cos597G/)、卡内基梅隆 (https://www.cs.cmu.edu/~leili/course/mldl22w/14-Transformer.pdf)等多所大学的课程中介绍



(https://www.llm-book.com)

更新：这篇文章现在已经变成了一本书！请查看 LLM-book.com (https://llm-book.com)，其中包含（第3章）更新和扩展版的文章，讲述了最新的变形金刚模型及其自原版变形金刚以来的七年演变（如多查询注意力和RoPE位置嵌入）。

在上一篇文章中，我们探讨了注意力 (https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/)——现代深度学习模型中无处不在的方法。注意力是一个帮助提升神经机器翻译应用性能的概念。在这篇文章中，我们将探讨**变形金刚**——一个利用注意力提升这些模型训练速度的模型。Transformer在特定任务中表现优于谷歌神经机器翻译模型。然而，最大的优势来自于《变形金刚》如何支持并行化。事实上，Google Cloud 建议使用 The Transformer 作为其 Cloud TPU (https://cloud.google.com/tpu/) 产品的参考模型。那么我们试着拆解这个模型，看看它是如何运作的。

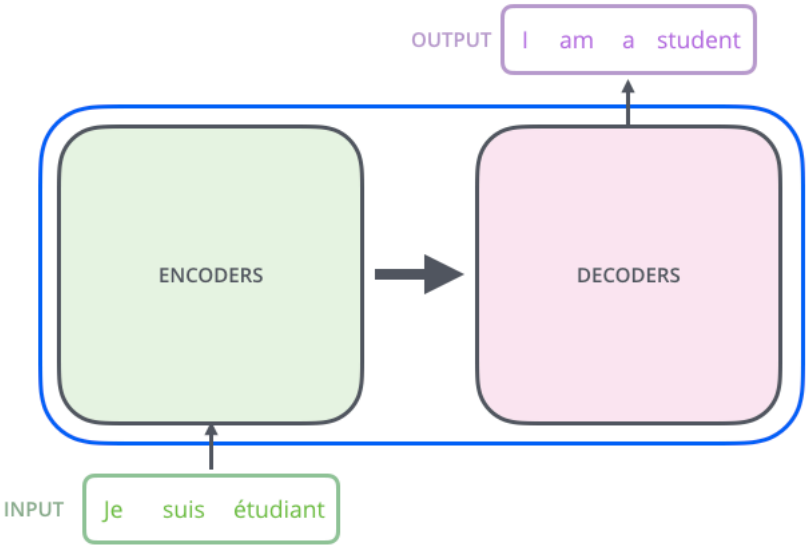
变压器是在论文《注意力就是你所需要的》 (https://arxiv.org/abs/1706.03762)中提出的。其 TensorFlow 实现作为 Tensor2Tensor (https://github.com/tensorflow/tensor2tensor) 包的一部分提供。哈佛的自然语言处理团队编写了一份指南，注释了论文中的PyTorch实现 (http://nlp.seas.harvard.edu/2018/04/03/attention.html)。在这篇文章中，我们将尝试稍微简化一些，逐一介绍这些概念，希望能让对该主题没有深入了解的人更容易理解。

2025年更新：我们制作了一个免费的简短课程 (https://bit.ly/4aRnn7Z)，内容与动画内容保持同步：

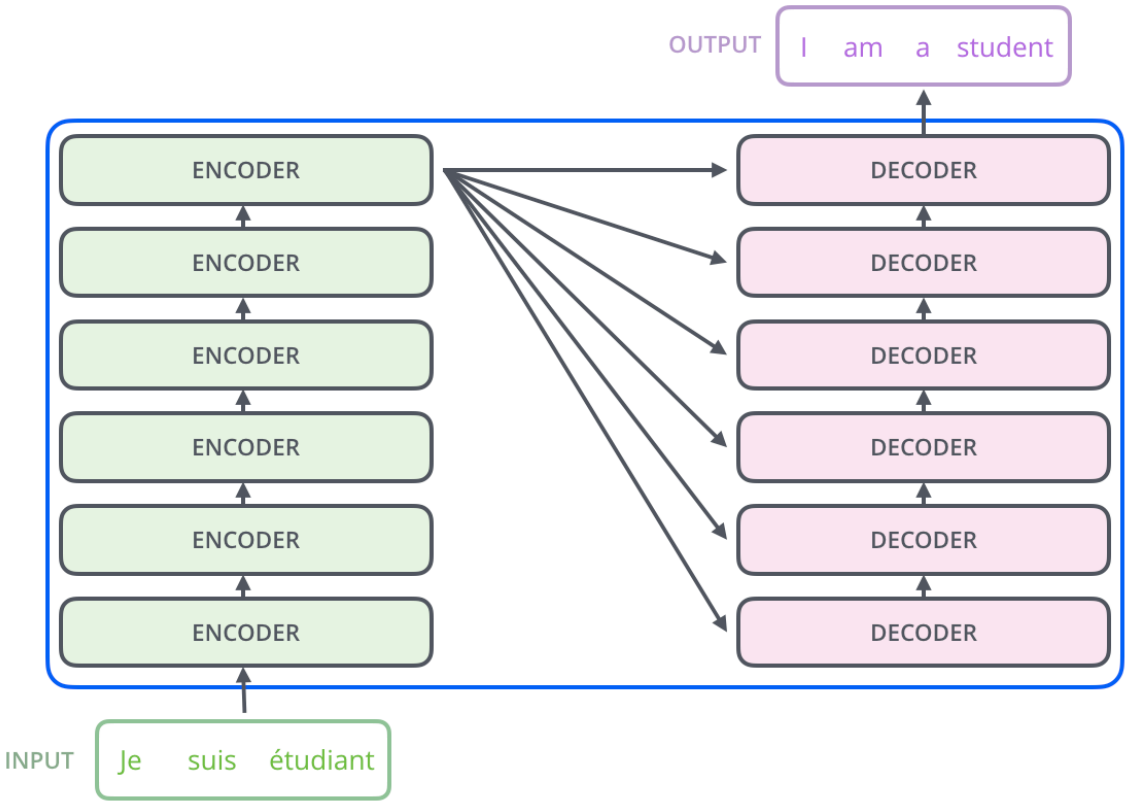
高层次观察

我们先把模型看作一个单一的黑盒子。在机器翻译应用中，它会将一种语言中的一句话输出到另一种语言中的句子。

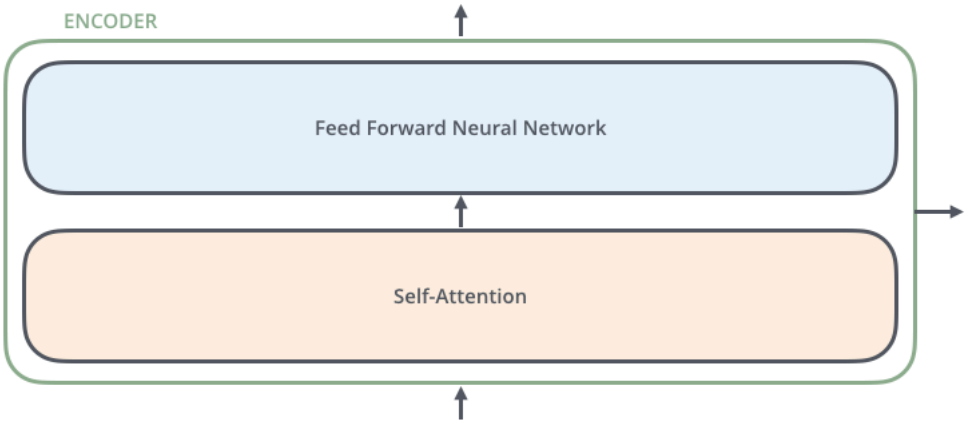
打开擎天柱的精髓，我们看到编码部分、解码部分以及它们之间的连接。



编码部分是一堆编码器（纸张上叠放了六个编码器——数字六没有什么神奇之处，完全可以尝试其他排列方式）。解码部分是同一编号的解码器堆栈。



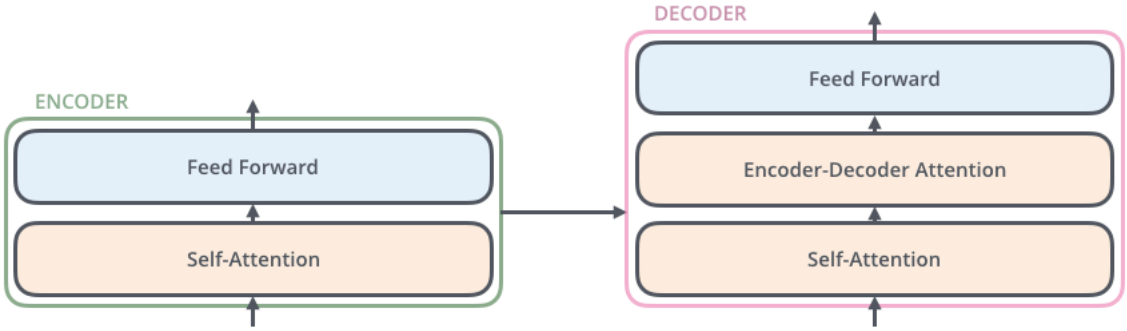
编码器结构相同（但权重不相同）。每个子层又分为两个子层：



编码器的输入首先流经自注意层——该层帮助编码器在编码特定词时观察输入句子中的其他单词。我们稍后会更深入地探讨自我关注。

自关注层的输出会被输入到前馈神经网络。完全相同的前馈网络独立应用于每个位置。

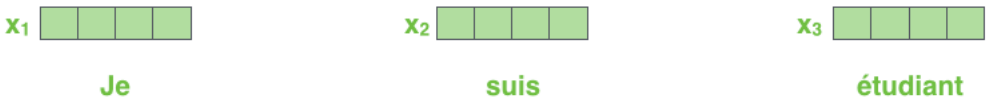
解码器包含这两层，但中间有一层注意力，帮助解码器专注于输入句子的相关部分（类似于 seq2seq 模型(https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/)中的注意力）。



将张量引入画面

既然我们已经看到了模型的主要组成部分，接下来让我们开始看看各种向量/张量，以及它们如何在这些分量之间流动，将训练好的模型的输入转化为输出。

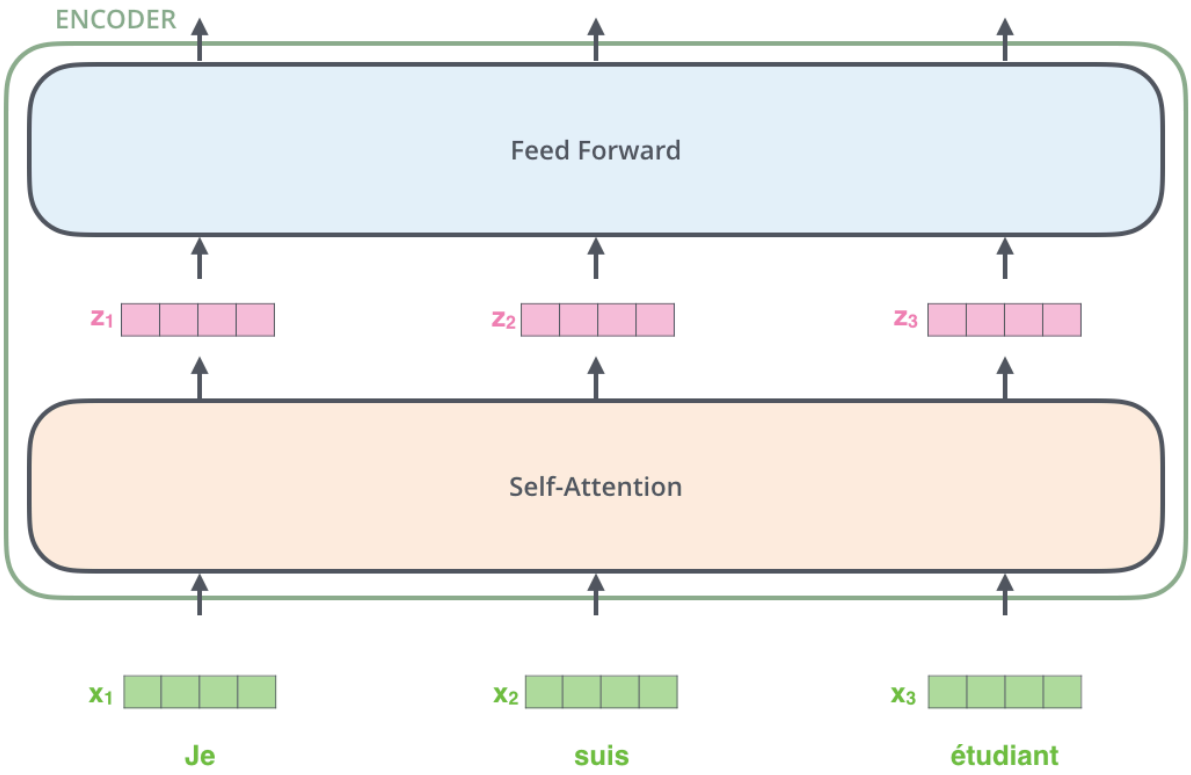
与一般NLP应用一样，我们首先用嵌入算法 (https://medium.com/deeper-learning/glossary-of-deep-learning-word-embedding-f90c3cec34ca)将每个输入词转换为向量。



每个词嵌入一个大小为512的向量中。我们将用这些简单的方框来表示这些向量。

嵌入只发生在最底的编码器中。所有编码器共有的抽象点是它们收到一份大小为512的向量列表——在底部编码器中，那是嵌入字，但在其他编码器中，则是编码器正下方的输出。这个列表的大小是我们可以设置的超参数——基本上就是训练数据集中最长句子的长度。

在将单词嵌入输入序列后，每个单词会流经编码器的两层。

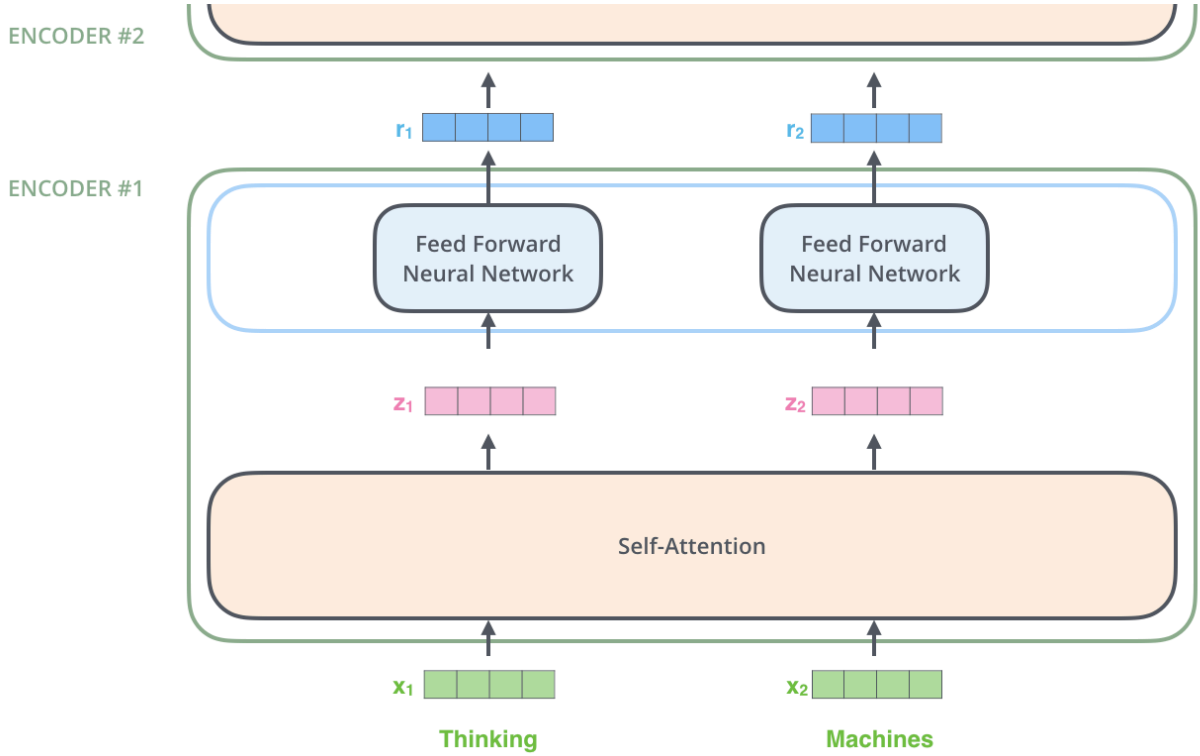


这里我们开始看到变换器的一个关键特性，即每个位置的字在编码器中流经自己的路径。在自我关注层中，这些路径之间存在依赖关系。然而，前馈层没有这些依赖关系，因此各种路径可以在流经前馈层的同时并行执行。

接下来，我们将把示例切换到一个更短的句子，看看编码器每个子层发生了什么。

现在我们开始编码了！

正如我们之前提到的，编码器接收的是一个向量列表作为输入。它通过将这些向量传递到“自关注”层，再进入前馈神经网络，然后将输出向上发送给下一个编码器来处理该列表。



每个位置的单词都经过自我关注过程。然后，它们都经过一个前馈神经网络——完全相同的网络，每个向量分别流经其中。

高层次的自我关注

别被我随意提到“自我关注”这个词当作每个人都应该熟悉的概念所迷惑。我个人直到读到《注意力就是你所需要的全部》论文之前，从未接触过这个概念。让我们来提炼一下它是如何运作的。

假设以下句子是我们想翻译的输入句子：

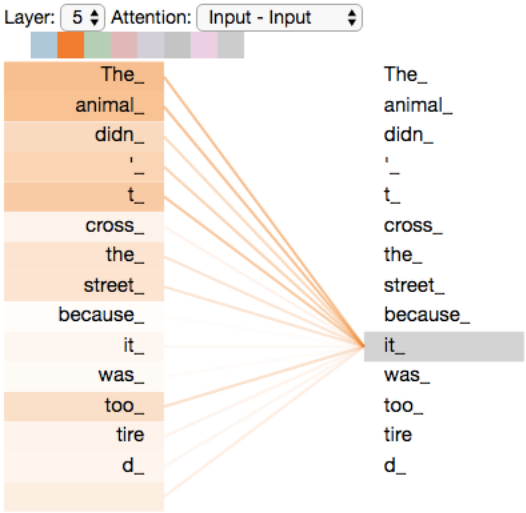
```
""The animal didn't cross the street because it was too tired
```

这句话中的“它”指的是什么？是指街道还是动物？对人类来说这是个简单的问题，但对算法来说却没那么简单。

当模型处理“它”这个词时，自我关注使其能够将“它”与“动物”联系起来。

当模型处理每个单词（输入序列中的每个位置）时，自我关注使它能够观察输入序列中的其他位置，寻找有助于更好地编码该单词的线索。

如果你熟悉RNN，可以想象一下，保持隐藏状态让RNN能够将其之前处理过的词语/向量的表示与当前处理的词语/向量结合起来。自我关注是变形金刚用来将“理解”其他相关词汇融入我们当前处理的词汇的方法。



当我们在编码器#5（堆栈中最顶的编码器）中编码“it”这个词时，注意力机制的一部分是聚焦于“动物”，并将其部分表示嵌入到“it”的编码中。

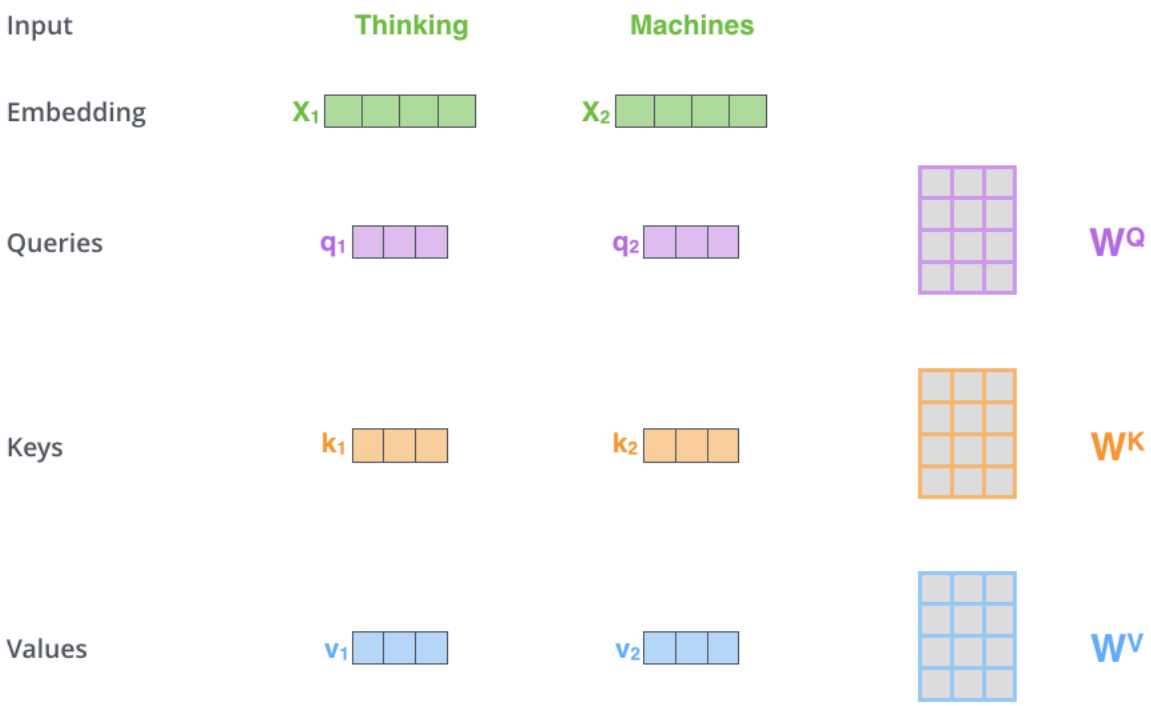
一定要查看Tensor2Tensor笔记本(https://colab.research.google.com/github/tensorflow/tensor2tensor/blob/master/tensor2tensor/notebooks/hello_t2t.ipynb)，那里可以加载变换器模型，并用这个交互式可视化进行分析。

细节自我关注

我们先看看如何用向量计算自我注意力，然后再看看实际实现方式——用矩阵。

计算自注意的第一**步**是从编码器的每个输入向量（此处为每个字的嵌入）创建三个向量。所以每个词我们创建一个查询向量、键向量和值向量。这些向量是通过将嵌入乘以我们在训练过程中训练的三个矩阵生成的。

注意这些新向量的维度比嵌入向量小。它们的维数为64，而嵌入和编码器输入/输出矢量的维数为512。它们不一定非得更小，这是一种架构选择，旨在使多头注意力的计算（大部分时间）保持恒定。



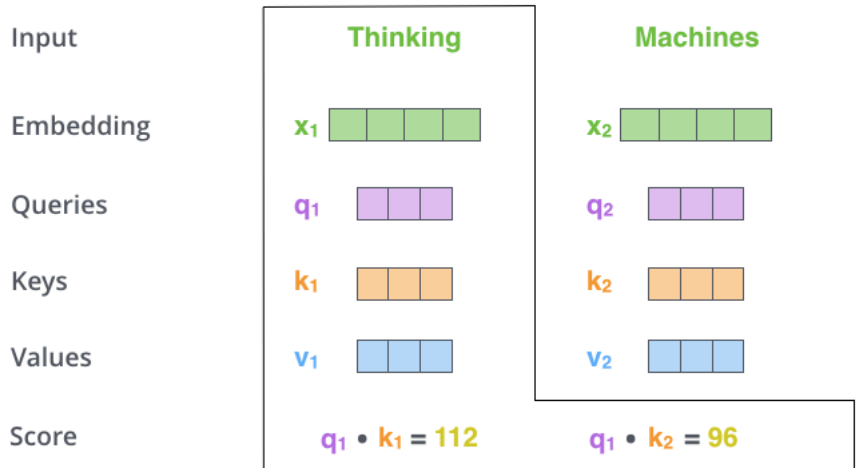
乘以 x1 与 WQ 权重矩阵相关联，得到 q1，即与该词相关联的“查询”向量。我们最终会为输入句中的每个单词创建一个“查询”、“键”和“值”投影。

“查询”、“键”和“值”向量是什么？

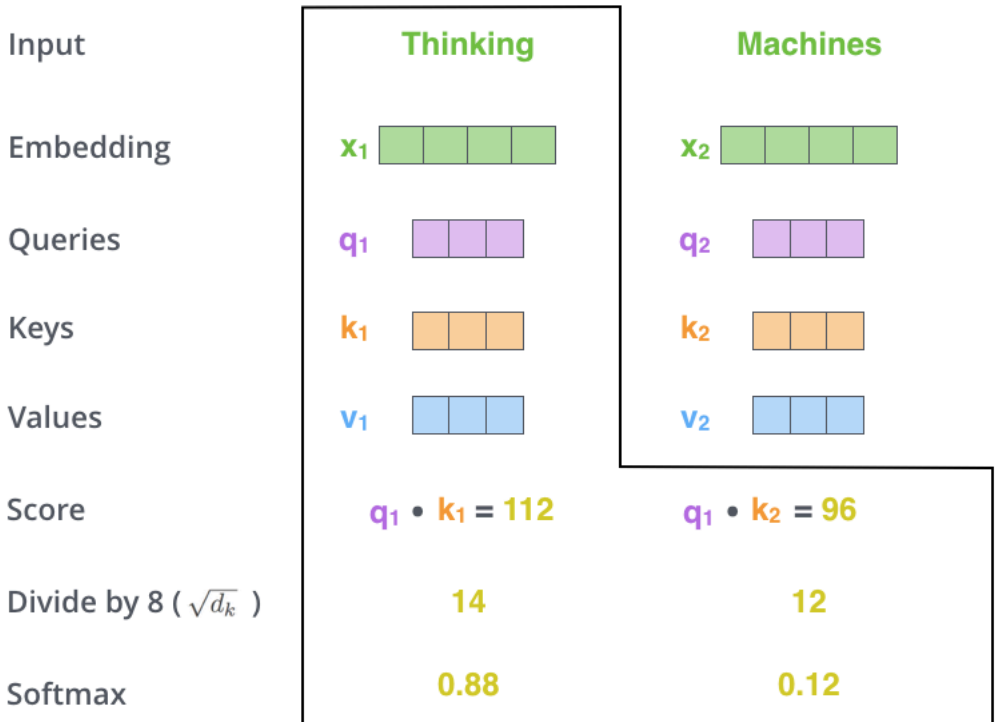
它们是用于计算和思考注意力的抽象概念。一旦你继续阅读下面注意力的计算方法，你就能大致了解这些向量所扮演的角色。

计算自我注意力的**第二步**是计算得分。假设我们正在计算这个例子中第一个词“思考”的自我注意力。我们需要对输入句中的每个单词进行评分。评分决定了在编码单词时，输入句子其他部分应给予多少关注。

得分是通过查询向量与我们评分的相应单词的键向量的点积计算得出的。所以如果我们处理位置#1的自注意，第一个分数就是q1和k1的点积。第二个分数是q1和k2的点积。



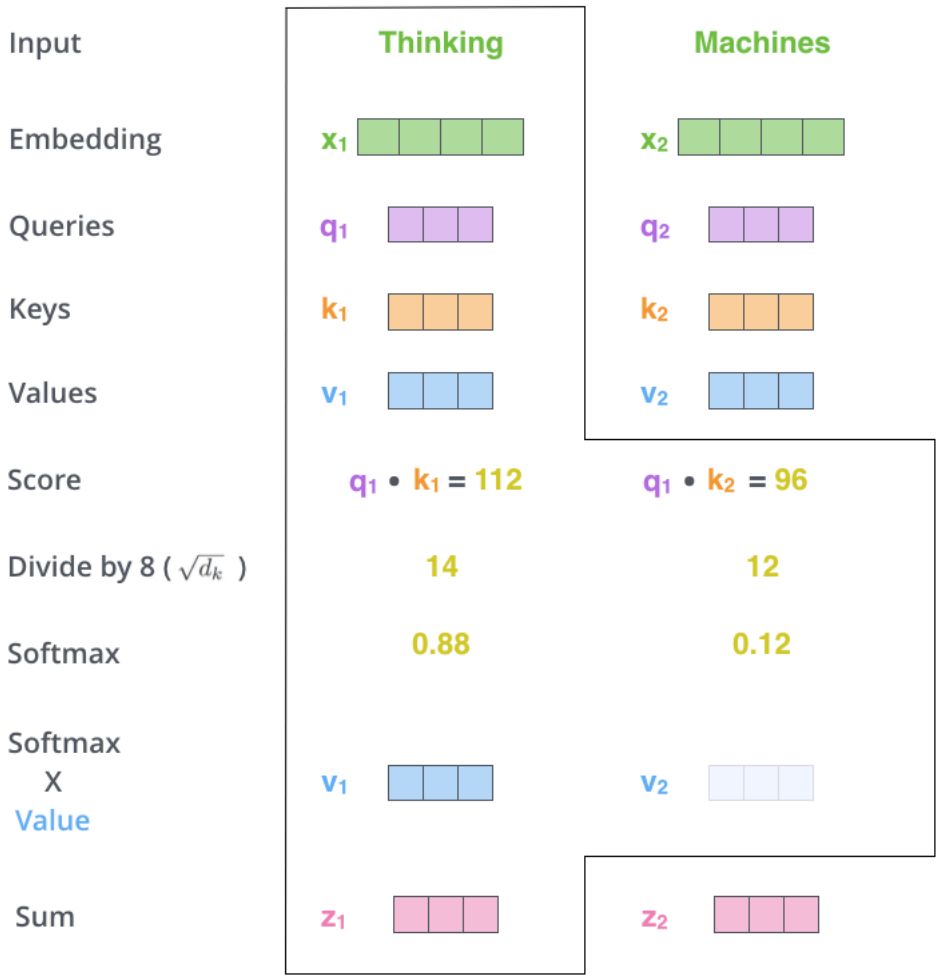
第三和第四步是将分数除以8（论文中使用的关键向量维数的平方根——64）。这导致梯度更稳定。这里可能还有其他可能的值，但这是默认的），然后将结果传递给softmax操作。Softmax会将分数归一化，使其全部为正，加起来为1。



该 softmax 分数决定每个单词在该位置的表达量。显然，这个位置的单词会有最高的 softmax 分数，但有时关注与当前单词相关的另一个单词也很有用。

第五步是将每个数值向量乘以软最大分（准备加总）。这里的直觉是保持我们想关注的单词的数值完整，并淹没无关的单词（比如乘以0.001这样的小数字）。

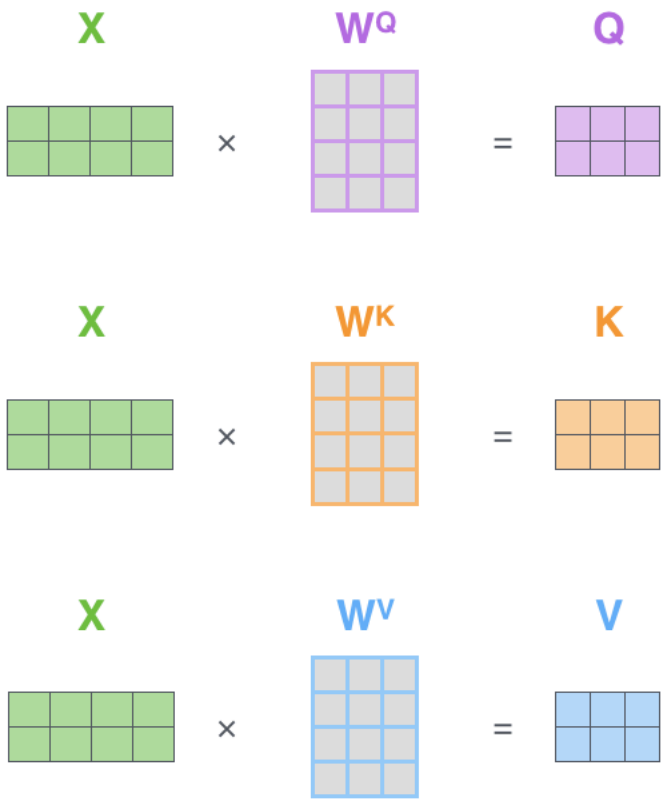
第六步是对加权值向量加总。这会在此位置（第一个词）产生自我注意力层的输出。



自我关注的计算到此结束。由此产生的向量可以发送到前馈神经网络。但在实际实现中，这种计算以矩阵形式进行，以加快处理速度。那么，既然我们已经看到了词层面计算的直觉，我们来看看这个问题。

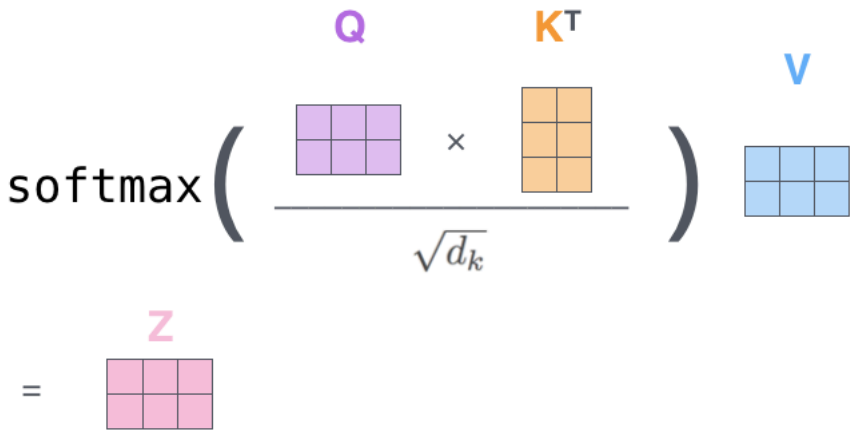
自我注意力的矩阵计算

第一步是计算查询矩阵、键矩阵和值矩阵。我们通过将嵌入打包到矩阵 X 中，并乘以我们训练的权重矩阵（WQ、WK、WV）来实现这一点。



X矩阵中的每一行对应输入句子中的一个词。我们再次看到嵌入向量大小的差异（512个，图中4个方格），与q/k/v向量（64个，或图中3个方格）的大小差异

最后，由于我们处理的是矩阵，我们可以将第二到第六步压缩到一个公式中，以计算自注意层的输出。

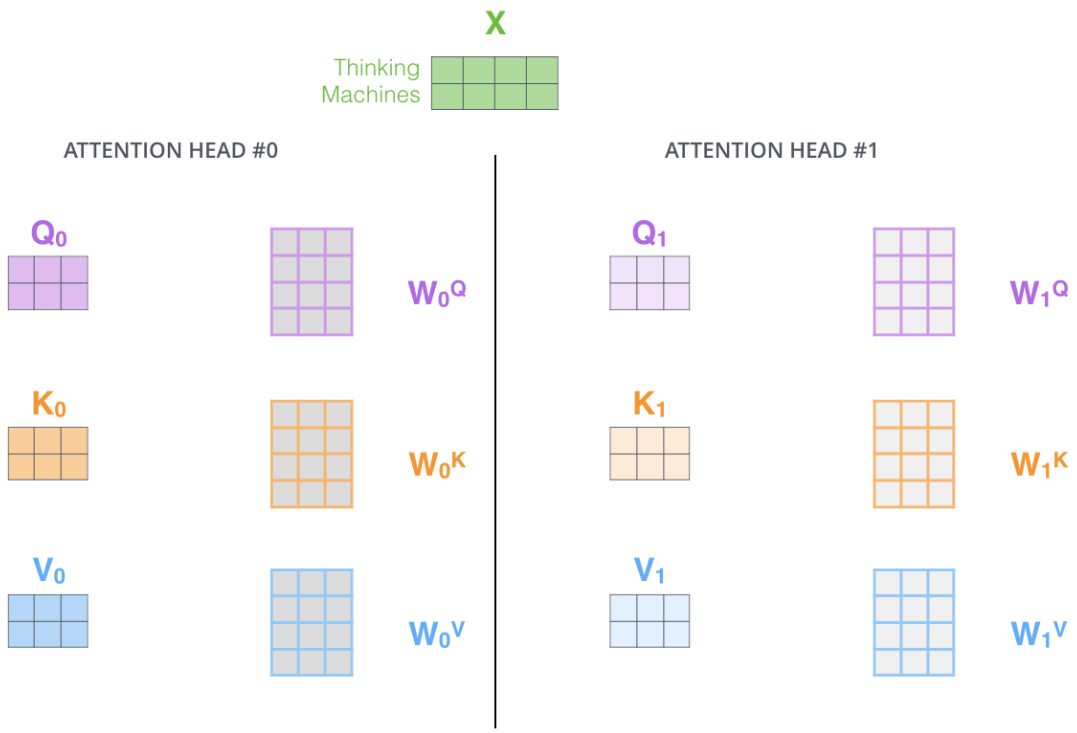


矩阵形式的自注意计算

多头兽

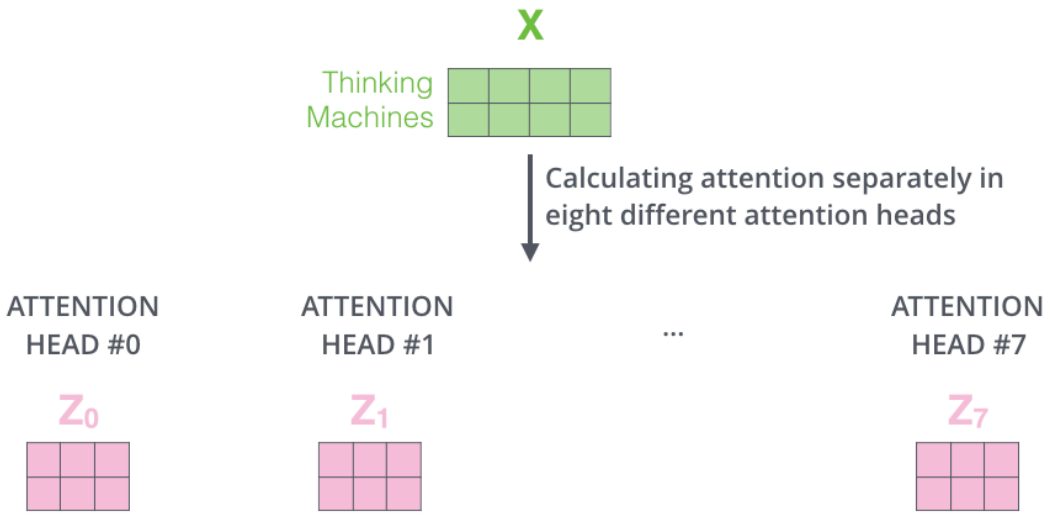
论文进一步完善了自我注意层，加入了一种称为“多头”注意力的机制。这通过两种方式提升了注意力层的性能：

1. 它扩展了模型对不同体位的专注能力。是的，在上面的例子中，z1包含了每一种编码的一点，但它可能被实际的单词本身所主导。如果我们翻译一句“动物没过马路，因为太累了”，知道“它”指哪个词会很有用。
2. 它为注意力层提供了多个“表示子空间”。正如我们接下来会看到的，多重注意力系统不仅有一个，而是多个查询/键/值权重矩阵（变换器使用八个注意力头，所以每个编码器/解码器总共有八组）。这些集合都是随机初始化的。然后，训练完成后，每个集合用来将输入嵌入（或来自较低编码器/解码器的向量）投影到不同的表示子空间中。



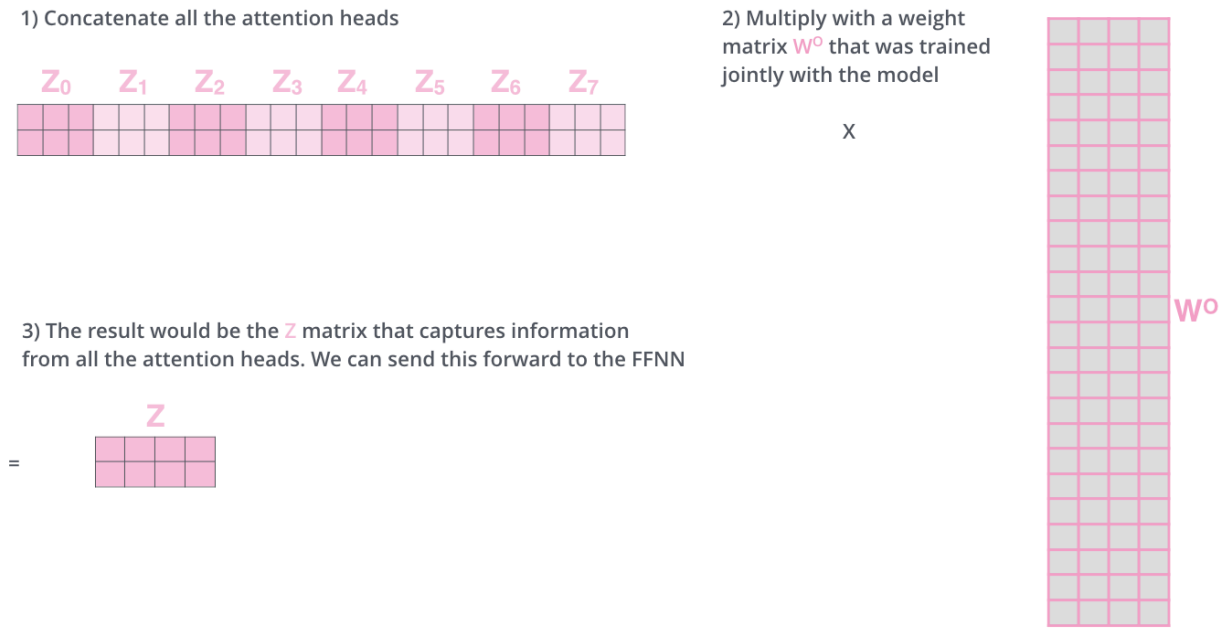
在多头注意力下，我们为每个头保持独立的Q/K/V权重矩阵，导致不同的Q/K/V矩阵。和之前一样，我们用X乘以WQ/WK/WV矩阵，得到Q/K/V矩阵。

如果我们用上面提到的自注意力计算，只用不同权重矩阵做八次，最终得到八个不同的Z矩阵

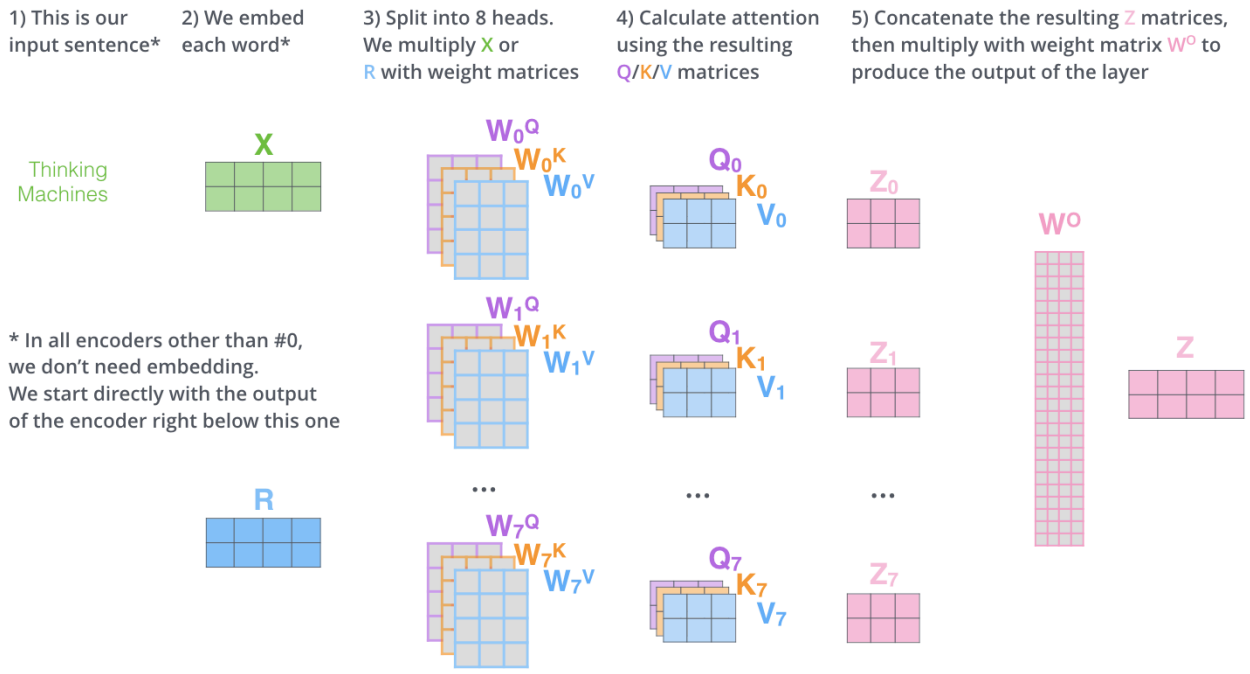


这给我们带来了一些挑战。前馈层并不期望有八个矩阵——它期望一个单一矩阵（每个词的向量）。所以我们需要一种方法，把这八个缩小成一个矩阵。

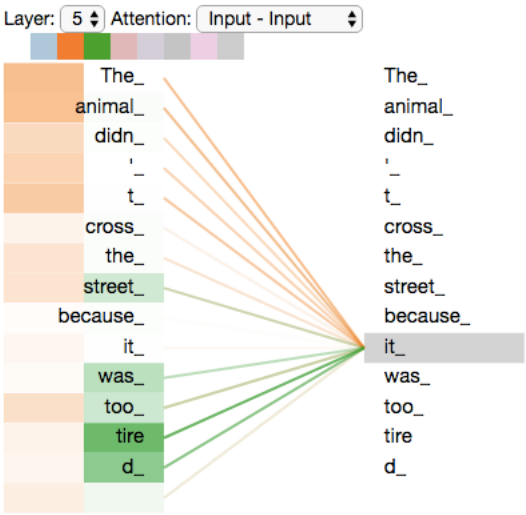
我们该怎么做？我们将矩阵串联，然后乘以额外的权重矩阵WO。



多头自我关注基本上就是这样。我意识到这涉及不少矩阵。让我尽量把它们都放在一个视觉图里，这样我们可以集中看到它们

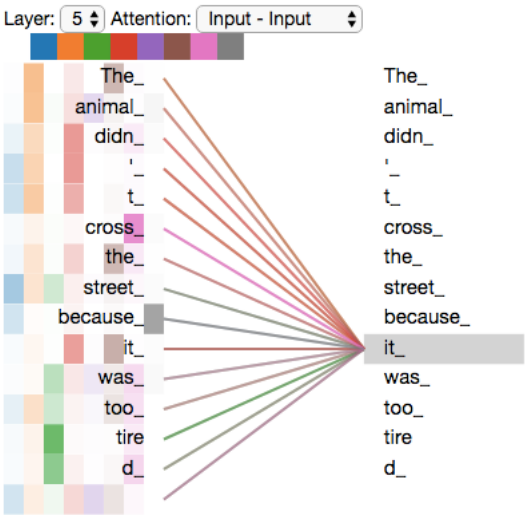


既然我们已经提到了注意力的头，让我们回顾之前的例子，看看在示例句中编码“它”这个词时，不同的注意力头关注的方向：



当我们编码“它”这个词时，一个注意力头最专注于“动物”，另一个注意力头则专注于“疲惫”——从某种意义上说，模型对“它”一词的表征融合了“动物”和“疲惫”的部分表征。

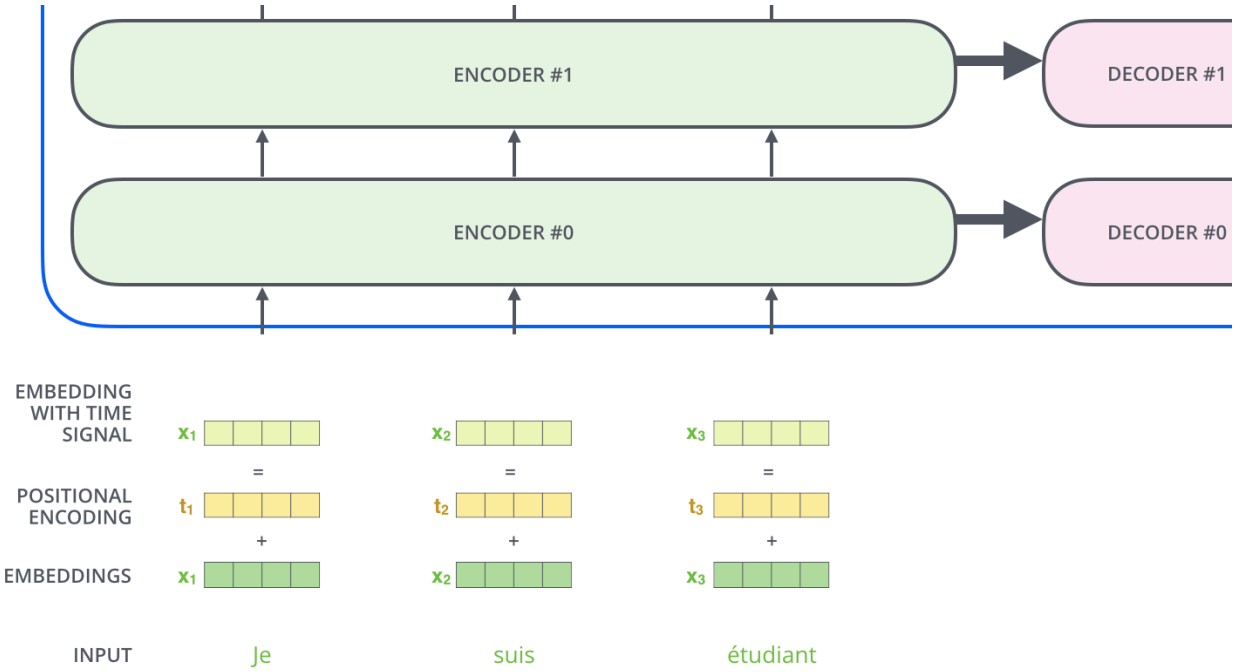
然而，如果把所有注意力的正面都加进去，情况就更难理解了：



使用位置编码表示序列的顺序

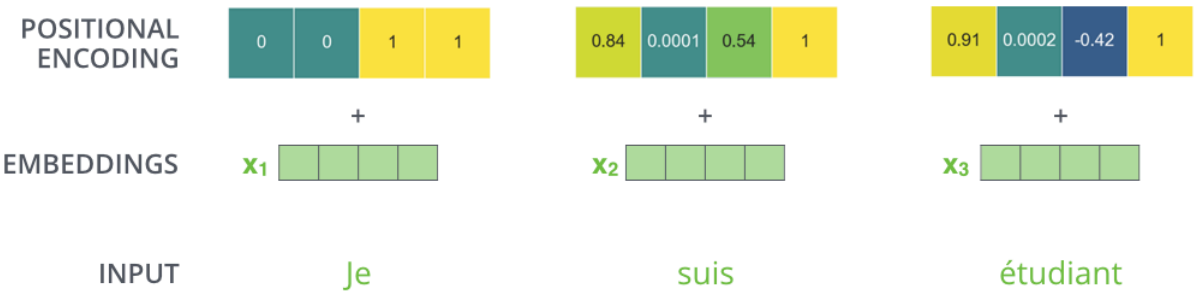
我们迄今为止描述的模型中缺少的一点是如何解释输入序列中单词的顺序。

为此，变换器会在每个输入嵌入中添加一个向量。这些向量遵循模型学习的特定模式，有助于确定每个词的位置，即序列中不同词之间的距离。这里的直觉是，将这些值添加到嵌入中，一旦嵌入向量投影到Q/K/V矢量中，并且在点积关注时，可以获得有意义的嵌入向量之间的距离。



为了让模型了解单词的顺序，我们添加了位置编码向量——其值遵循特定的模式。

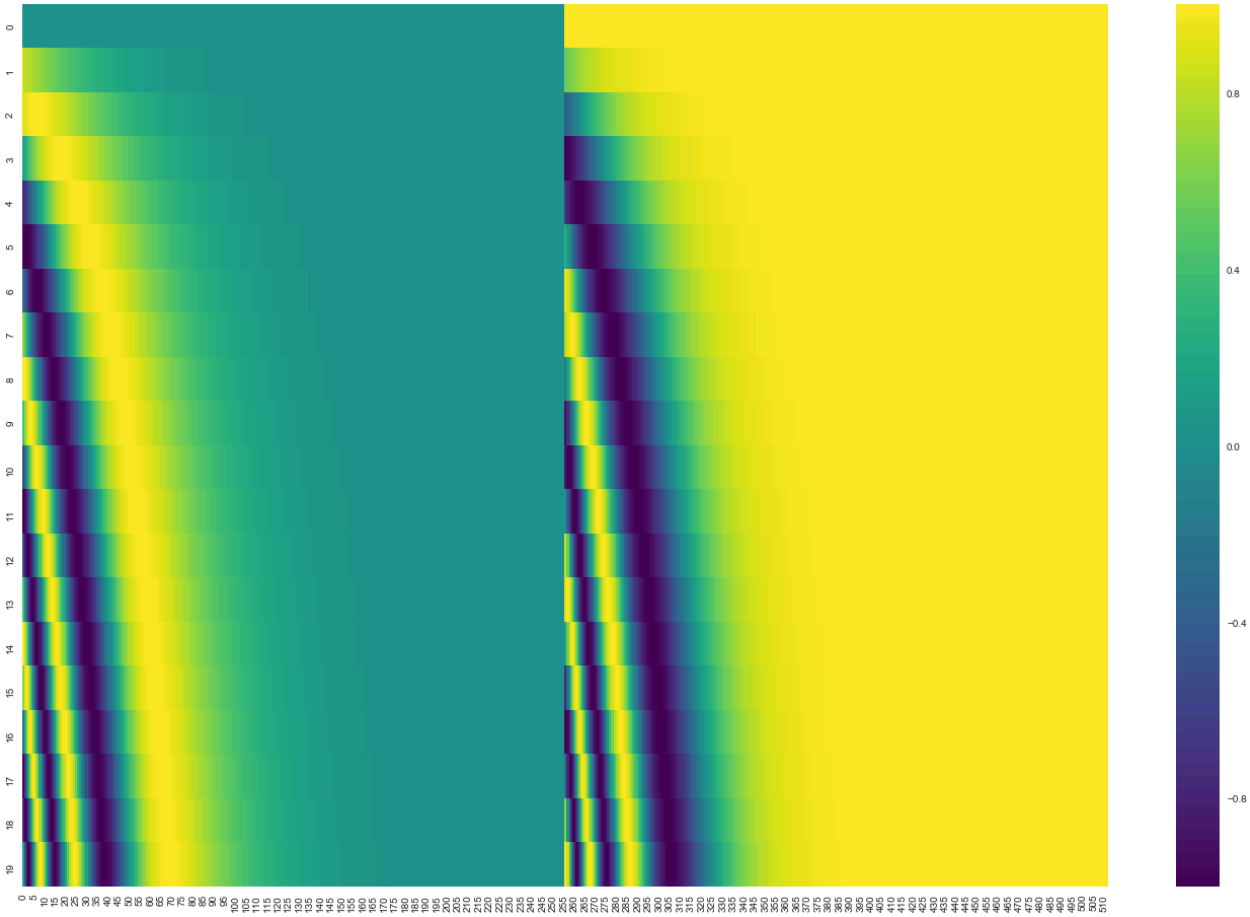
如果假设嵌入的维数为4，实际的位置编码会是这样的：



一个真实的定位编码示例，玩具嵌入尺寸为4

这个模式会是什么样子？

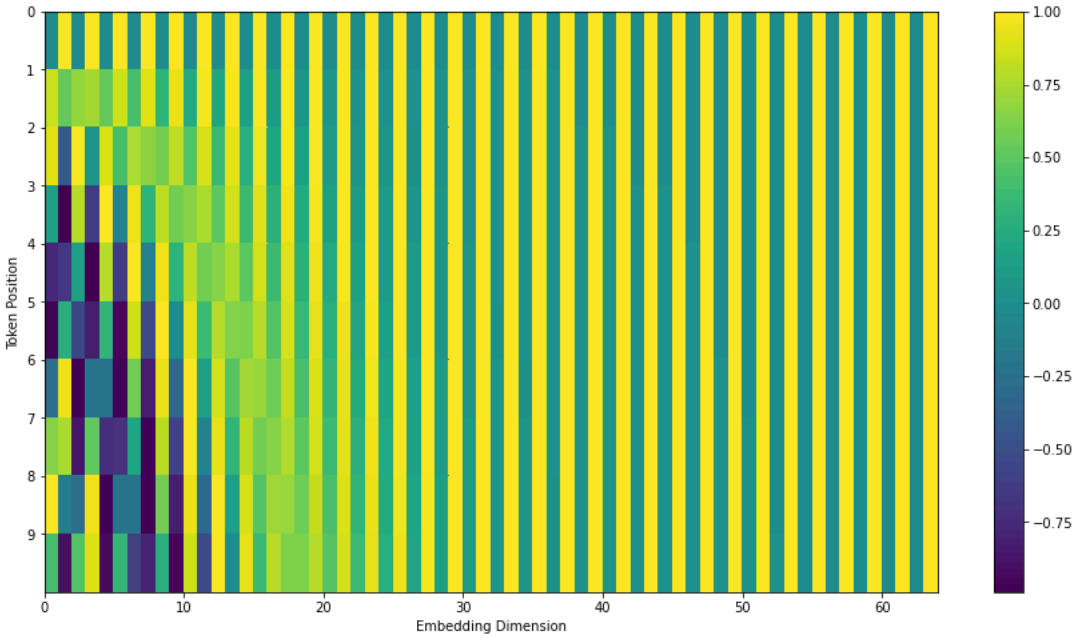
在下图中，每一行对应一个向量的位置编码。所以第一行就是我们要加到输入序列中第一个词嵌入的向量。每行包含512个值——每个值在1到-1之间。我们用颜色编码了它们，这样图案就能看见。



这是20个字（行）位置编码的真实例子，嵌入大小为512列。你可以看到它在中心被劈成两半。这是因为左半部分的值由一个函数（使用正弦函数）生成，右半部分由另一个函数（使用余弦）生成。然后它们被串联起来，形成每个位置编码向量。

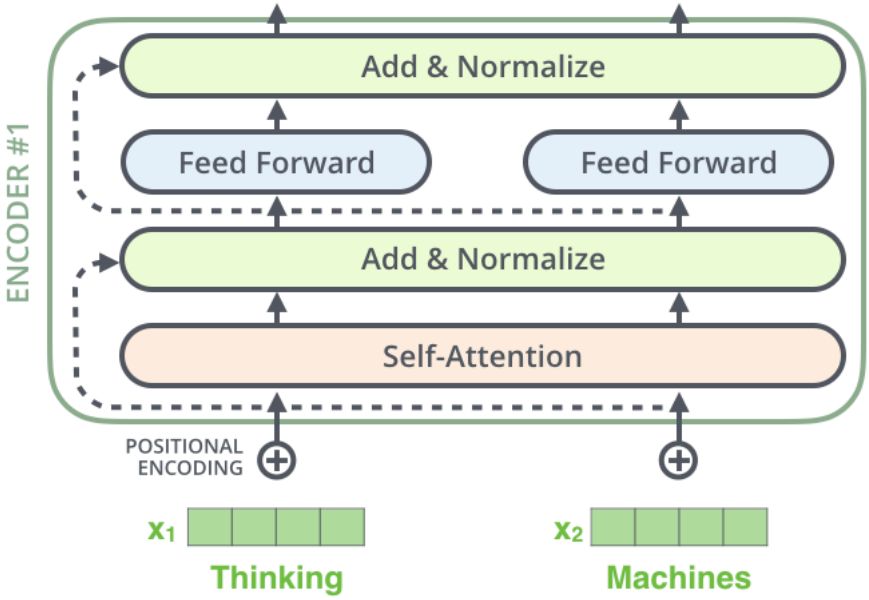
位置编码的公式见论文（第3.5节）。你可以在`get_timing_signal_1d()`(https://github.com/tensorflow/tensor2tensor/blob/23bd23b9830059fbc349381b70d9429b5c40a139/tensor2tensor/layers/comm中看到生成位置编码的代码。这并不是唯一可能的定位编码方法。不过，它带来了一个优势，就是能够扩展到未见的序列长度（例如，如果我们的训练模型被要求翻译比训练集中任何句子都长的句子）。

2020年7月更新：上面显示的位置编码来自变换器的张量2张量实现。论文中展示的方法稍有不同，它不是直接连接，而是将两个信号交织在一起。下图展示了这种状况。这是生成它的代码(https://github.com/jalammar/jalammar.github.io/blob/master/notebookes/transformer/transformer_positional_encoding_graph.ip

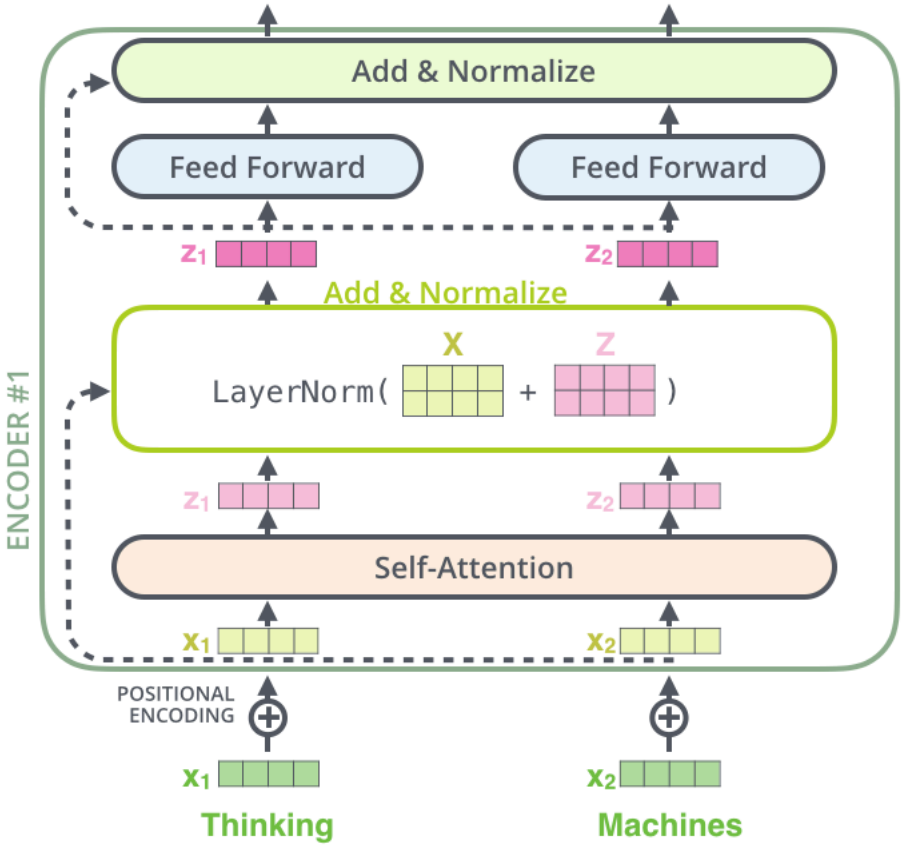


残差

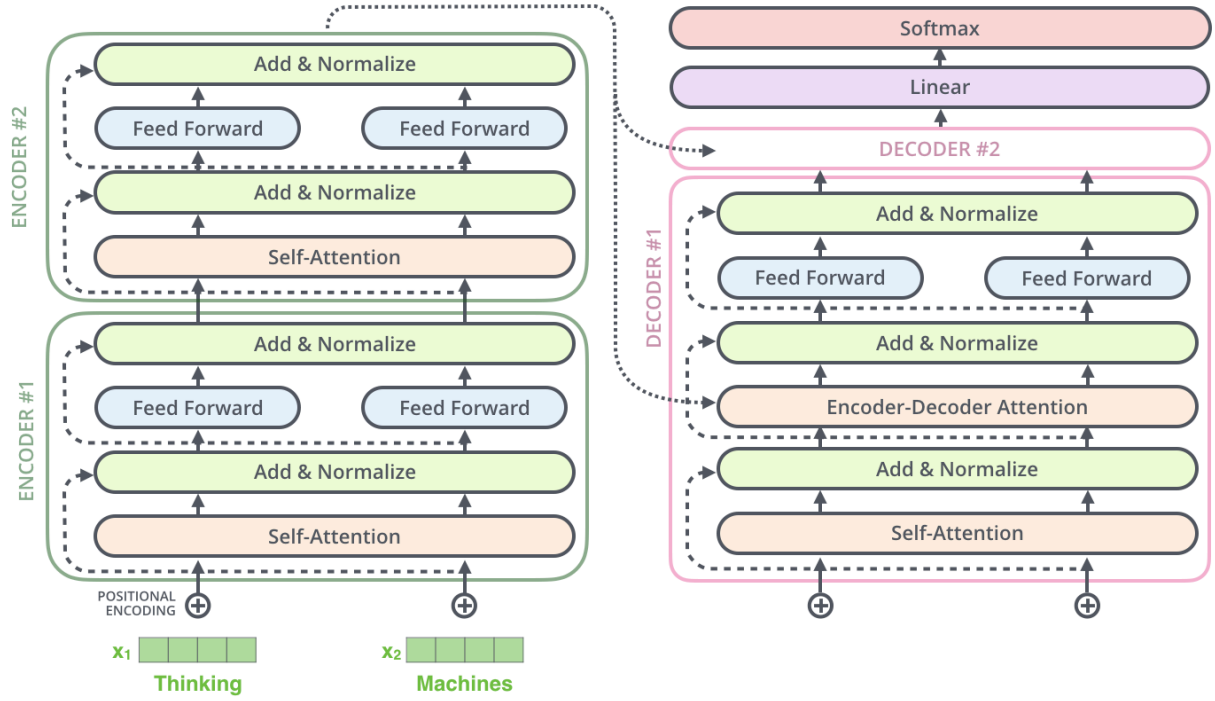
在继续之前，我们需要提及编码器架构中的一个细节是，每个编码器的每个子层（自关注，ffnn）都有一个残余连接环绕其周围，随后是层归一 (https://arxiv.org/abs/1607.06450)化步骤。



如果我们要可视化与自关注相关的向量和层范数操作，它会是这样的：



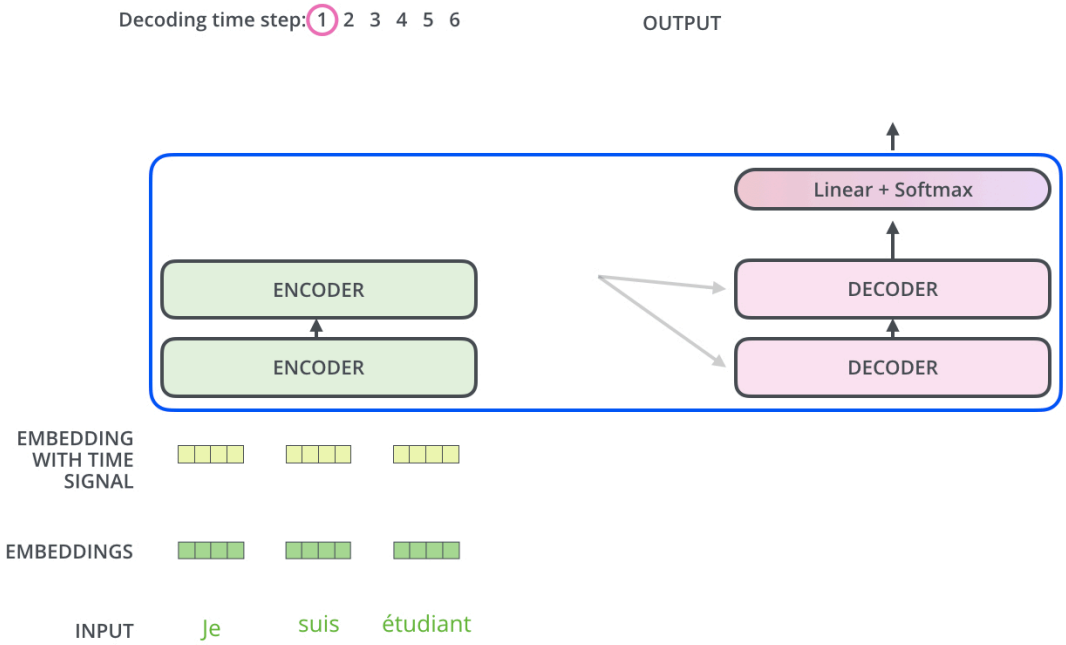
解码器的子层也是如此。如果我们要想象一个由两个叠加编码器和解码器的变换器组成的变压器，它会是这样的：



解码器一面

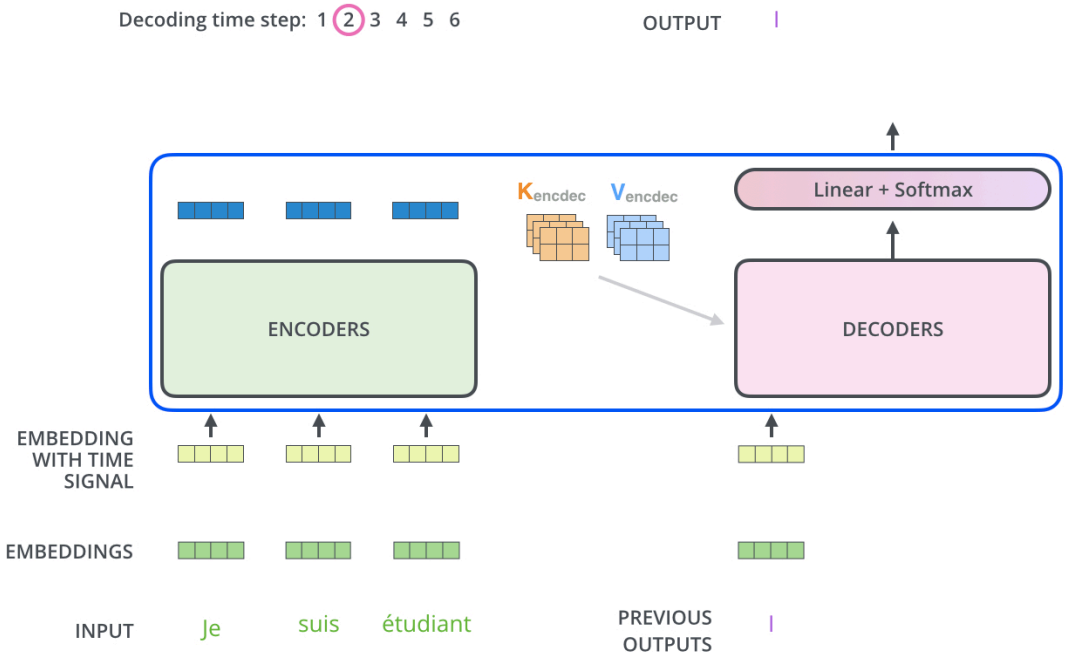
现在我们已经涵盖了编码器方面大部分概念，基本上也知道解码器的组件是如何工作的。但让我们看看它们是如何协同工作的。

编码器首先处理输入序列。顶部编码器的输出随后被转换为一组注意力向量K和V。这些内容应被每个解码器在其“编码器-解码器注意”层中使用，帮助解码器聚焦输入序列中的适当位置：



编码阶段结束后，我们开始解码阶段。解码阶段的每一步都会输出输出序列中的一个元素（此处为英文翻译句子）。

接下来的步骤重复此过程，直到遇到一个特殊符号，表示变换器解码器已完成输出。每一步的输出会在下一个时间步输入到底部解码器，解码器会像编码器一样气泡化解码结果。就像我们对编码器输入所做的一样，我们会在解码器输入中嵌入并添加位置编码，以指示每个字的位置。



解码器中的自注意层的工作方式与编码器中的略有不同：

在解码器中，自注意层只允许关注输出序列中较早的位置。这是通过在自注意计算的软最大步骤之前，将未来位置屏蔽（设置为 ）。`-inf`

“编码器-解码器注意”层的工作原理类似于多头自注意层，但它从下一层创建查询矩阵，并从编码器栈的输出中提取键和值矩阵。

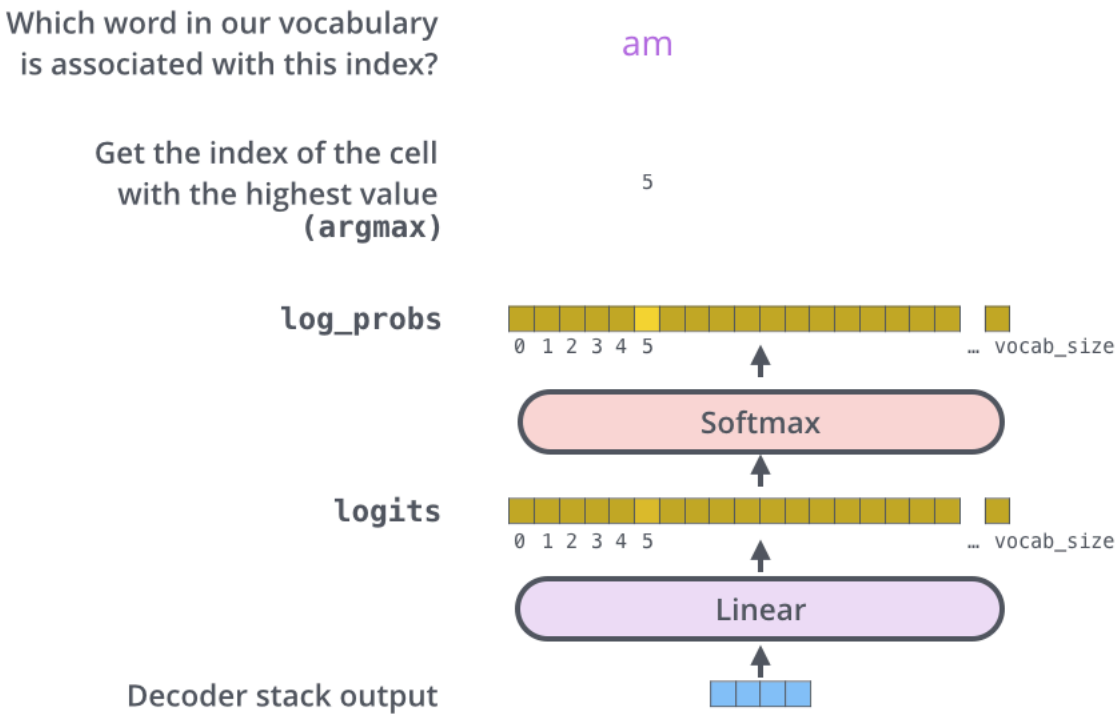
最终线性层和软极大层

解码器栈输出一个浮点向量。我们怎么把它变成一个词？这就是最后一个线性层的工作，随后是软极大层。

线性层是一个简单的全连通神经网络，将解码器堆栈产生的向量投影为一个更大的向量，称为logit向量。

假设我们的模型知道1万个独特的英语单词（模型的“输出词汇”），这些词汇是它从训练数据集中学习到的。这使得logit向量宽度为10,000格——每个格子对应一个唯一单词的得分。这就是我们解释模型输出和线性层的方式。

softmax层随后将这些分数转换为概率（均为正数，加总为1.0）。选择概率最高的单元格，并生成与之相关的词作为该时间步的输出。



该图从底部开始，向量作为解码器栈的输出。然后它被转化为输出字。

训练回顾

既然我们已经涵盖了通过训练过的变换器实现的整个前向传递过程，不妨快速了解一下训练模型的直觉。

在训练中，未训练的模型会经过完全相同的前传。但由于我们是在带标签的训练数据集上训练的，我们可以将其输出与实际正确输出进行比较。

为了直观地理解这一点，假设我们的输出词汇只包含六个单词（“a”、“am”、“i”、“thanks”、“student”和“<eos>”(句末的缩写)）。

Output Vocabulary

WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5

我们模型的输出词汇是在训练开始前的预处理阶段创建的。

一旦定义了输出词汇，就可以使用相同宽度的向量来表示词汇中的每个词。这也称为单热编码。例如，我们可以通过以下向量表示单词“am”：

Output Vocabulary

WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5

One-hot encoding of the word “am”

0.0	1.0	0.0	0.0	0.0	0.0

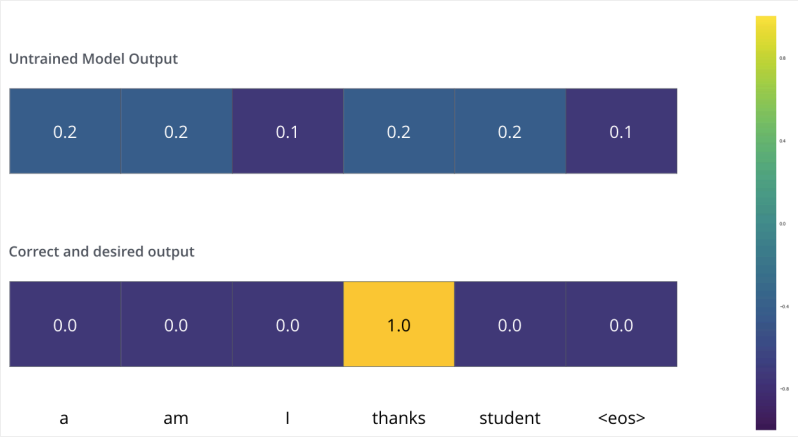
示例：输出词汇的单热编码

在本次回顾之后，我们来讨论模型的损失函数——我们在训练阶段优化的指标，以最终形成一个训练有素且希望极其准确的模型。

损失函数

假设我们正在训练模型。假设这是我们培训阶段的第一步，我们用一个简单的例子来训练——把“merci”翻译成“thanks”。

这意味着我们希望输出是表示“thanks”这个词的概率分布。但由于该模型尚未被训练，这种情况暂时不太可能实现。

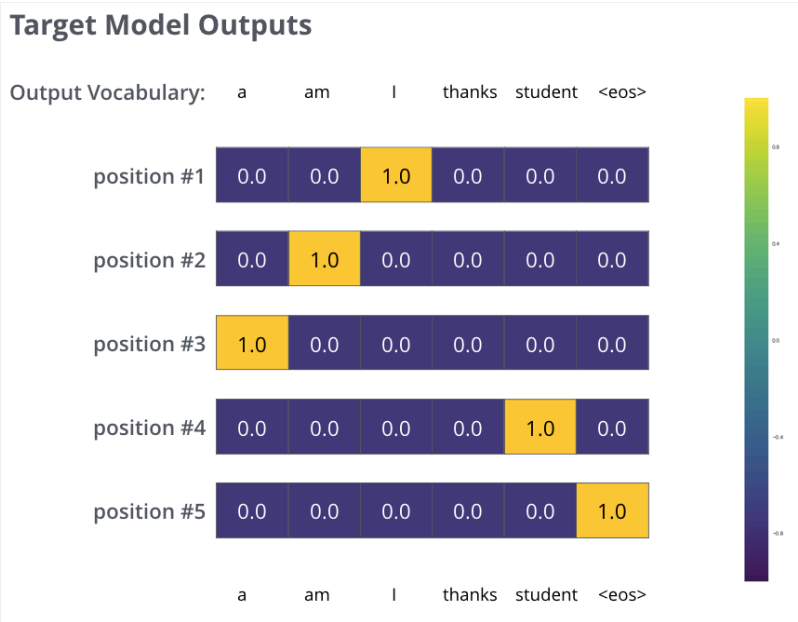


由于模型的参数（权重）都是随机初始化的，（未训练的）模型会为每个单元格/单词生成任意值的概率分布。我们可以将其与实际输出比较，然后利用反向传播调整模型的所有权重，使输出更接近期望的输出。

如何比较两种概率分布？我们只需从其中一个中减去另一个。更多细节请参见交叉熵(https://colah.github.io/posts/2015-09-Visual-Information/)和库尔巴克–莱布勒散度(https://www.countbayesie.com/blog/2017/5/9/kullback-leibler-divergence-explained)。

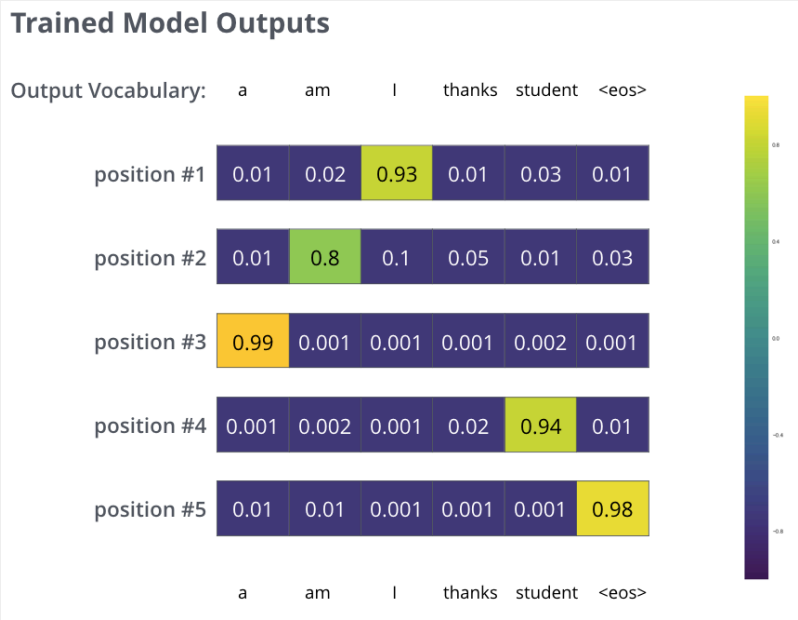
但请注意，这只是一个过于简化的例子。更现实的是，我们会用比一个词更长的句子。例如——输入：“je suis étudiant”，期望输出：“我是学生”。这实际上意味着，我们希望模型能够依次输出概率分布，其中：

- 每个概率分布都用宽度为vocab_size的向量表示（在我们的玩具例子中是6，但更现实地是像30,000或50,000这样的数字）
- 第一个概率分布在与“i”相关的单元格中概率最高
- 第二种概率分布在与“am”一词相关的单元格中概率最高
- 如此循环，直到第五个输出分布显示“"符号，该符号也关联着来自10,000元素词汇表的单元格。`<end of sentence>`



我们将在训练示例中训练模型的目标概率分布，针对一个示例句子。

在足够大的数据集上训练模型足够长时间后，我们希望生成的概率分布会是这样的：



希望训练后模型能输出我们预期的正确平移。当然，这并不意味着这句话是否属于训练数据集（参见：交叉验证 (https://www.youtube.com/watch?v=TIgfjmp-4BA)）。注意每个位置都会获得一点概率，即使它不太可能是该时间步的输出——这是softmax非常有用的特性，有助于训练过程。

现在，由于模型一次输出一个，我们可以假设模型选择概率分布中概率最高的词，丢弃其余的。这是一种做法（叫贪心解码）。另一种方法是保留顶部两个词（比如“I”和“a”），然后在下一步运行模型两次：一次假设第一个输出位置是单词“I”，另一次假设第一个输出位置是单词“a”，保留错误较少的版本，考虑同时存在#1和#2两个位置。我们对位置#2和#3重复此法.....等等。这种方法称为“束搜索”，在我们的例子中，beam_size为2（意味着始终有两个部分假设（未完成的平译）被存储在内存中，top_beams也是2（意味着返回两个平移）。这些都是你可以尝试的超参数。

前进，改变

希望你觉得这里是打破僵局的有用起点，了解变形金刚的主要概念。如果你想更深入，我建议接下来的步骤：

- 阅读《注意力就是你所需要 (https://arxiv.org/abs/1706.03762)的全部》论文、Transformer博客文章（《变换器：一种用于语言理解的新颖神经网络架构 (https://ai.googleblog.com/2017/08/transformer-novel-neural-network.html)》）以及Tensor2Tensor的公告 (https://ai.googleblog.com/2017/06/accelerating-deep-learning-research.html)。
- 观看Łukasz Kaiser (https://www.youtube.com/watch?v=rBCqOTEfxvg)讲解模型及其细节
- 使用作为Tensor2Tensor仓库一部分提供的Jupyter Notebook (https://colab.research.google.com/github/tensorflow/tensor2tensor/blob/master/tensor2tensor/notebooks/hello_t2t.ipynb)进行游戏
- 可以探索 Tensor2Tensor 仓库 (https://github.com/tensorflow/tensor2tensor)。

后续作品：

- 神经机器翻译的深度可分卷积 (https://arxiv.org/abs/1706.03059)
- 一个模型 学习所有这些 (https://arxiv.org/abs/1706.05137)
- 序列模型用的离散自编码器 (https://arxiv.org/abs/1801.09797)
- 通过总结长序列生成维基百科 (https://arxiv.org/abs/1801.10198)
- 图像转换器 (https://arxiv.org/abs/1802.05751)
- 变压器模型的培训技巧 (https://arxiv.org/abs/1804.00247)
- 带有相对位置表示的自我关注 (https://arxiv.org/abs/1803.02155)
- 使用离散潜在变量的序列模型快速解码 (https://arxiv.org/abs/1803.03382)
- Adafactor：带有亚线性记忆成本的自适应学习率 (https://arxiv.org/abs/1804.04235)

致谢

感谢Illia Polosukhin (https://twitter.com/ilblackdragon)、Jakob Uszkoreit (http://jakob.uszkoreit.net/)、Llion Jones (https://www.linkedin.com/in/llion-jones-9ab3064b)、Lukasz Kaiser (https://ai.google/research/people/LukaszKaiser)、Niki Parmar (https://twitter.com/nikiparmar09)和Noam Shazeer (https://dblp.org/pers/hd/s/Shazeer:Noam)对本文早期版本的反馈。

欢迎在推特 (https://twitter.com/JayAlammar)上联系我，寻求任何纠正或反馈。
撰写于2018年6月27日

Language Models & Co.

Large language models, their internals, and applications.

By Jay Alammar • Over 32,000 subscribers

Type your email... Subscribe

By subscribing you agree to Substack's Terms of Use, our Privacy Policy and our Information collection notice

substack

(http://creativecommons.org/licenses/by-nc-sa/4.0/)
本作品采用知识共享署名-非商业性-相同方式共享4.0国际许可 (http://creativecommons.org/licenses/by-nc-sa/4.0/)协议授权。
归因示例：
Alammar， J（2018）。图解变形金刚【博客文章】。取自 https://jalammar.github.io/illustrated-transformer/ (https://jalammar.github.io/illustrated-transformer/)

注：如果你翻译了任何帖子，请告诉我，我会把你的翻译链接到原帖。我的邮箱在关于页面 (/about)。

(https://github.com/jalammar) (https://www.linkedin.com/in/jalammar) (https://www.twitter.com/jayalammar)